

Hibernate Query Language (HQL)

1. Introduction to HQL

What is HQL?

Hibernate Query Language (HQL) is an object-oriented query language provided by Hibernate for querying data stored in relational databases. Unlike SQL, which operates directly on database tables and columns, HQL works with **Java classes (entities)** and their **attributes (properties)**.

HQL is database-independent, meaning the same query can run on different databases such as Oracle, MySQL, PostgreSQL, SQL Server, etc. Hibernate internally translates HQL statements into the appropriate SQL dialect of the underlying database.

Why HQL?

Without HQL:

```
SELECT EMP_NAME FROM EMPLOYEE WHERE SALARY > 50000;
```

The above SQL is database-specific and uses table names.

With HQL:

```
FROM Employee WHERE salary > 50000
```

Here,

- **Employee** → Java Entity Class
- **salary** → Java instance variable

Hibernate automatically converts this into SQL.

Advantages of HQL

- Database Independent
- Object-Oriented Query Language
- Works with Java Classes
- Easy to Learn
- Supports CRUD Operations
- Supports Aggregate Functions

- Supports Joins
 - Supports Pagination
 - Supports Named Parameters
 - Prevents SQL Injection (when using parameters)
-

Sample Entity

Assume we have the following entity.

```
@Entity
@Table(name="EMPLOYEE")
public class Employee {

    @Id
    private int empId;

    private String empName;

    private String department;

    private double salary;

    private String city;
}
```

Sample Database

empId	empName	department	salary	city
101	Rahul	IT	65000	Delhi
102	Priya	HR	45000	Noida
103	Amit	IT	80000	Delhi
104	Neha	Sales	52000	Mumbai
105	Rohan	IT	90000	Delhi

HQL Syntax

FROM EntityName

General Syntax

```
SELECT propertyName
FROM EntityName
WHERE condition
ORDER BY propertyName
```

Example 1 : Retrieve All Employees

HQL

```
String hql = "FROM Employee";
```

Execution

```
Session session = factory.openSession();

Query<Employee> query =
    session.createQuery(hql, Employee.class);

List<Employee> list = query.list();

for(Employee e : list)
{
    System.out.println(e.getEmpName());
}

session.close();
```

Equivalent SQL

```
SELECT * FROM EMPLOYEE;
```

Example 2 : Retrieve Specific Columns

```
String hql =
"SELECT empName, salary FROM Employee";
```

Execution

```
Query<Object[]> query =
    session.createQuery(hql, Object[].class);

List<Object[]> list = query.list();

for(Object row[] : list)
{
    System.out.println(row[0]+" "+row[1]);
}
```

SQL

```
SELECT EMP_NAME, SALARY
```

```
FROM EMPLOYEE;
```

Example 3 : WHERE Clause

Retrieve employees having salary greater than 60000.

```
String hql =  
"FROM Employee WHERE salary > 60000";
```

SQL

```
SELECT *  
FROM EMPLOYEE  
WHERE SALARY>60000;
```

Example 4 : AND Operator

```
String hql =  
"FROM Employee WHERE department='IT' AND salary>70000";
```

Example 5 : OR Operator

```
String hql =  
"FROM Employee WHERE city='Delhi' OR city='Noida'";
```

Example 6 : ORDER BY

Ascending

```
String hql =  
"FROM Employee ORDER BY salary";
```

Descending

```
String hql =  
"FROM Employee ORDER BY salary DESC";
```

Example 7 : Aggregate Functions

COUNT()

```
String hql =  
"SELECT COUNT(*) FROM Employee";
```

Output

MAX()

```
String hql =  
"SELECT MAX(salary) FROM Employee";
```

Output

90000

MIN()

```
String hql =  
"SELECT MIN(salary) FROM Employee";
```

Output

45000

AVG()

```
String hql =  
"SELECT AVG(salary) FROM Employee";
```

Output

66400

SUM()

```
String hql =  
"SELECT SUM(salary) FROM Employee";
```

Output

332000

Example 8 : GROUP BY

```
String hql =  
"SELECT department, COUNT(*) FROM Employee GROUP BY department";
```

Output

Department Count

IT	3
----	---

Department Count

HR 1

Sales 1

Example 9 : HAVING Clause

```
String hql =  
"SELECT department,AVG(salary) FROM Employee GROUP BY department HAVING  
AVG(salary)>60000";
```

Example 10 : Named Parameters

Never write

```
String hql =  
"FROM Employee WHERE empId=101";
```

Instead,

```
String hql =  
"FROM Employee WHERE empId=:id";  
Query<Employee> query =  
session.createQuery(hql,Employee.class);
```

```
query.setParameter("id",101);
```

```
Employee emp=query.uniqueResult();
```

Advantages

- Secure
 - Prevents SQL Injection
 - Dynamic Queries
-

Example 11 : LIKE Operator

```
String hql =  
"FROM Employee WHERE empName LIKE 'R%'";
```

Output

```
Rahul  
Rohan
```

Example 12 : BETWEEN Operator

```
String hql =  
"FROM Employee WHERE salary BETWEEN 50000 AND 80000";
```

Example 13 : IN Operator

```
String hql =  
"FROM Employee WHERE city IN('Delhi','Mumbai')";
```

Example 14 : DELETE Query

```
Transaction tx =  
session.beginTransaction();  
  
String hql =  
"DELETE FROM Employee WHERE empId=105";  
  
Query query=  
session.createQuery(hql);  
  
query.executeUpdate();  
  
tx.commit();
```

Example 15 : UPDATE Query

```
Transaction tx =  
session.beginTransaction();  
  
String hql =  
"UPDATE Employee SET salary=85000 WHERE empId=103";  
  
Query query=  
session.createQuery(hql);  
  
query.executeUpdate();  
  
tx.commit();
```

Example 16 : Pagination

Retrieve first 5 records.

```
Query<Employee> query =  
session.createQuery("FROM Employee", Employee.class);  
  
query.setFirstResult(0);  
  
query.setMaxResults(5);  
  
List<Employee> list=query.list();
```

Example 17 : INNER JOIN

Assume

Department $\longleftrightarrow$ Employee

```
String hql=
"SELECT e.empName,d.departmentName FROM Employee e INNER JOIN e.department
d";
```

Example 18 : Fetch Join

```
String hql=
"FROM Employee e JOIN FETCH e.department";
```

Avoids LazyInitializationException.

Example 19 : DISTINCT

```
String hql=
"SELECT DISTINCT department FROM Employee";
```

Example 20 : Named Query

```
@NamedQuery(
name="Employee.findAll",
query="FROM Employee")
```

Usage

```
Query<Employee> query =
session.createNamedQuery(
"Employee.findAll",
Employee.class);
```

SQL vs HQL

SQL

HQL

Works on Tables

Works on Java Classes

Uses Column Names

Uses Instance Variables

Database Dependent

Database Independent

SQL	HQL
Returns Records	Returns Objects
Uses Table Name	Uses Entity Name

Important Interview Questions

Q1. What is HQL?

Answer: Hibernate Query Language is an object-oriented query language that operates on Hibernate entities instead of database tables.

Q2. Difference between SQL and HQL?

SQL	HQL
Table	Entity
Column	Property
Record	Object
DB Specific	Database Independent

Q3. Difference between `list()` and `uniqueResult()` ?

<code>list()</code>	<code>uniqueResult()</code>
Returns List	Returns Single Object
Multiple Records	Exactly One Record

Q4. What is `executeUpdate()` ?

Used for

- UPDATE
- DELETE
- INSERT (HQL supports insert-select, not direct INSERT VALUES)

Returns the number of affected rows.

Q5. Why use named parameters?

```
FROM Employee WHERE empId=:id
```

Advantages:

- Prevents SQL Injection
 - Improves readability
 - Supports dynamic parameter binding
-

Laboratory Exercise

Create an `Employee` entity and perform the following HQL operations:

1. Display all employees.
2. Display employees with a salary greater than ₹50,000.
3. Update the salary of an employee using HQL.
4. Delete an employee record based on `empId`.
5. Count the total number of employees.
6. Find the highest and lowest salary.
7. Group employees by department and display the count.
8. Retrieve employees whose names start with 'A'.
9. Display the first five employee records using pagination.
10. Use named parameters to search for an employee by `empId`.

These examples provide students with practical exposure to the most commonly used HQL features in enterprise Java applications.